

Software Requirements Specification

Automated Web Data Acquisition and Preprocessing System for LLM Training

Version	1.0
Authors	Abhay Gugale, Aanoush Surana
Organization	Indian Institute of Information Technology, Pune
Date	23 August 2026

Software Requirements Specification

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Project Scope
 - 1.5 References
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Features
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
- 3. System Features
 - 3.1 Scraping Job Configuration
 - 3.2 Web Page Retrieval and Parsing
 - 3.3 Data Extraction and Preprocessing
 - 3.4 URL Deduplication
 - 3.5 Result Storage and Export
 - 3.6 Error Handling and Logging
- 4. External Interface Requirements
 - 4.1 User Interfaces
 - 4.2 Hardware Interfaces
 - 4.3 Software Interfaces
 - 4.4 Communications Interfaces
- 5. Other Nonfunctional Requirements
 - 5.1 Performance Requirements
 - 5.2 Safety Requirements
 - 5.3 Security Requirements
 - 5.4 Software Quality Attributes
- 6. Other Requirements
 - 6.1 Development Methodology

Appendix A: Glossary

Appendix B: Analysis Models

Appendix C: Issues List

1. Introduction

1.1 Purpose

The purpose of this project is to develop an **Automated Web Data Acquisition and Preprocessing System for LLM Training**. The system automates the collection and preprocessing of web data by retrieving webpages, discovering additional pages, extracting useful content, removing irrelevant content and duplicate URLs, and producing structured JSON data.

1.2 Document Conventions

- **Must/Should:** Required behaviour.
- **Should:** Desired behaviour.
- **TBD:** Requirement not yet finalized.
- **Scraping Job:** One execution involving one or more URLs.

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, testers, reviewers, evaluators, and system users. Sections 1 and 2 provide an overview; Section 3 defines system behaviour; Section 4 defines interfaces; and Section 5 defines nonfunctional requirements.

1.4 Project Scope

The system will:

- Accept and validate HTTP/HTTPS URLs.
- Crawl webpages with configurable depth and optional recursive crawling.
- Support basic JavaScript-rendered pages.
- Extract text, titles, headings, hyperlinks, tables, and image references.
- Remove irrelevant or noisy content.
- Detect and skip duplicate URLs.
- Download encountered images and PDFs.
- Provide job status and error reporting.
- Store processed results in JSON.

The initial version will not support user authentication, CAPTCHA or access-control bypass, paywall bypass, or scheduled crawling. Scheduled crawling is planned for a future version.

1.5 References

- Project requirements and problem statement.
- Supplied SRS template.
- Documentation for the selected HTTP, parsing, and rendering libraries.

2. Overall Description

2.1 Product Perspective

The system is a web application that automates web data acquisition and preprocessing.

Workflow: URLs → Retrieval → Crawling → Extraction → Preprocessing → Deduplication → JSON

2.2 Product Features

- Scraping job configuration
- Webpage retrieval and parsing
- Configurable crawling
- Text extraction and preprocessing
- URL deduplication
- Image/PDF downloading
- JSON generation
- Error handling and logging

2.3 User Classes and Characteristics

Normal User: Provides URLs, configures jobs, and accesses results.

Data Analyst/Researcher: Uses the system for automated data collection and NLP/LLM-related datasets.

2.4 Operating Environment

- Web application
- Windows/Linux
- General-purpose computer or server
- Internet connectivity
- Sufficient storage

2.5 Design and Implementation Constraints

- Extraction quality depends on webpage structure.
- JavaScript-heavy websites may have limited support.
- The system will not bypass access restrictions.
- robots.txt will not be used as a crawling restriction.
- JSON is the initial output format.
- Scheduling is outside Version 1.

2.6 User Documentation

Documentation will cover installation, usage, configuration, output, errors, and limitations.

2.7 Assumptions and Dependencies

The system assumes accessible webpages, network connectivity, and sufficient storage.

3. System Features

3.1 Scraping Job Configuration

Priority: High

Stimulus/Response: User provides URLs and settings → system validates them and creates the scraping job.

REQ-F-001: The system shall accept one or more target URLs.

REQ-F-002: The system shall validate target URLs.

REQ-F-003: The system shall allow crawl depth to be configured.

REQ-F-004: The system shall allow recursive crawling to be enabled or disabled.

REQ-F-005: The system shall allow users to start and monitor scraping jobs.

3.2 Web Page Retrieval and Parsing

Priority: High

Stimulus/Response: System receives a valid URL → retrieves and parses the webpage.

REQ-F-006: The system shall retrieve webpages using HTTP/HTTPS.

REQ-F-007: The system shall parse retrieved HTML.

REQ-F-008: The system shall support basic JavaScript-rendered pages.

REQ-F-009: The system shall discover hyperlinks when recursive crawling is enabled.

REQ-F-010: The system shall apply the configured crawl depth.

REQ-F-011: The system shall use request timeouts and report retrieval/parsing failures.

3.3 Data Extraction and Preprocessing

Priority: High

Stimulus/Response: Webpage is parsed → system extracts and preprocesses its content.

REQ-F-012: The system shall extract titles, headings, relevant text, hyperlinks, image references, and basic HTML tables.

REQ-F-013: The system shall remove irrelevant or noisy content.

REQ-F-014: The system shall retain processed textual content rather than raw HTML.

REQ-F-015: The system shall download encountered images and PDF files.

REQ-F-016: The system shall not generate unavailable information.

3.4 URL Deduplication

Priority: High

Stimulus/Response: A URL is discovered → system checks whether it has already been processed and skips it if duplicated.

REQ-F-017: The system shall maintain a record of processed URLs.

REQ-F-018: The system shall skip duplicate URLs.

REQ-F-019: URL normalization is TBD.

3.5 Result Storage and Export

Priority: High

Stimulus/Response: Processing is completed → system generates structured JSON output.

REQ-F-020: The system shall generate JSON output.

REQ-F-021: Output shall contain the source URL, processed text, and relevant metadata.

REQ-F-022: Output shall indicate scraping status where applicable.

3.6 Error Handling and Logging

Priority: High

Stimulus/Response: System encounters an error → records it and reports the corresponding URL/job status.

REQ-F-023: The system shall detect invalid URLs and failed requests.

REQ-F-024: The system shall record parsing and extraction errors.

REQ-F-025: The system shall provide success/failure status for processed URLs.

REQ-F-026: Individual webpage failures shall not unnecessarily terminate the entire job.

The exact extraction and preprocessing mechanisms are TBD.

Content-level duplicate detection is outside Version 1.

The exact JSON schema is TBD.

4. External Interface Requirements

4.1 User Interfaces

- Enter URLs.
- Configure crawling options.
- Start and monitor jobs.
- Access results.
- View errors and status.

4.2 Hardware Interfaces

No specialized hardware is required. A general-purpose computer or server with sufficient storage and network connectivity is sufficient.

4.3 Software Interfaces

- HTTP/HTTPS requests
- HTML parsing
- JavaScript rendering
- Data extraction
- JSON generation
- Image/PDF downloading

Exact technologies are TBD.

4.4 Communications Interfaces

HTTP/HTTPS shall be used for communication with target web servers. Timeouts and controlled request pacing shall be applied.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

REQ-NF-001: The system should process webpages within a reasonable time.

REQ-NF-002: Request timeouts shall prevent indefinite waiting.

REQ-NF-003: Requests shall be controlled when processing multiple URLs.

Specific performance targets are TBD.

5.2 Safety Requirements

REQ-NF-004: The system should avoid excessive requests.

REQ-NF-005: Request pacing should be used.

REQ-NF-006: The system shall not bypass CAPTCHA, authentication, paywalls, or other access restrictions.

The crawler will not use robots.txt as a crawling restriction.

5.3 Security Requirements

REQ-NF-007: User-provided URLs shall be validated.

REQ-NF-008: Sensitive information shall not be stored unnecessarily.

REQ-NF-009: Errors shall not expose unnecessary internal information.

REQ-NF-010: The system shall not bypass access-control mechanisms.

REQ-NF-011: User authentication is not required in Version 1.

5.4 Software Quality Attributes

- **Reliability:** Individual webpage failures should not stop an entire job.
- **Maintainability:** Major components should be modular.
- **Usability:** Job configuration and result access should be straightforward.
- **Testability:** Major components should be independently testable.
- **Extensibility:** Future features such as scheduling and additional output formats should be supportable.

6. Other Requirements

6.1 Development Methodology

The project will use the **Agile SDLC model** because it supports incremental development, adaptation to different webpage structures, continuous testing, and early development of a working prototype.

Waterfall and V-Model: Less flexible for iterative changes.

Spiral: Would introduce unnecessary complexity for this academic project.

Appendix A: Glossary

Term	Definition
Web Scraper	System that retrieves and extracts information from webpages.
Crawler	Component that discovers webpages by following links.
Scraping Job	One execution of the scraping process.
Seed URL	Initial URL from which crawling begins.
Crawl Depth	Maximum level of link-following allowed.
Data Preprocessing	Cleaning and preparing extracted content.
URL Deduplication	Identifying and skipping already processed URLs.
JSON	Structured data format used for output.
LLM	Large Language Model.
NLP	Natural Language Processing.

Appendix B: Analysis Models

No analysis models are included in Version 1.

Appendix C: Issues List

- Exact technology stack.
- Text extraction and preprocessing mechanism.
- URL normalization mechanism.
- JSON schema.
- JavaScript rendering mechanism.
- Storage implementation.
- Retry and rate-limiting policies.
- Performance targets.
- PDF text extraction behaviour.